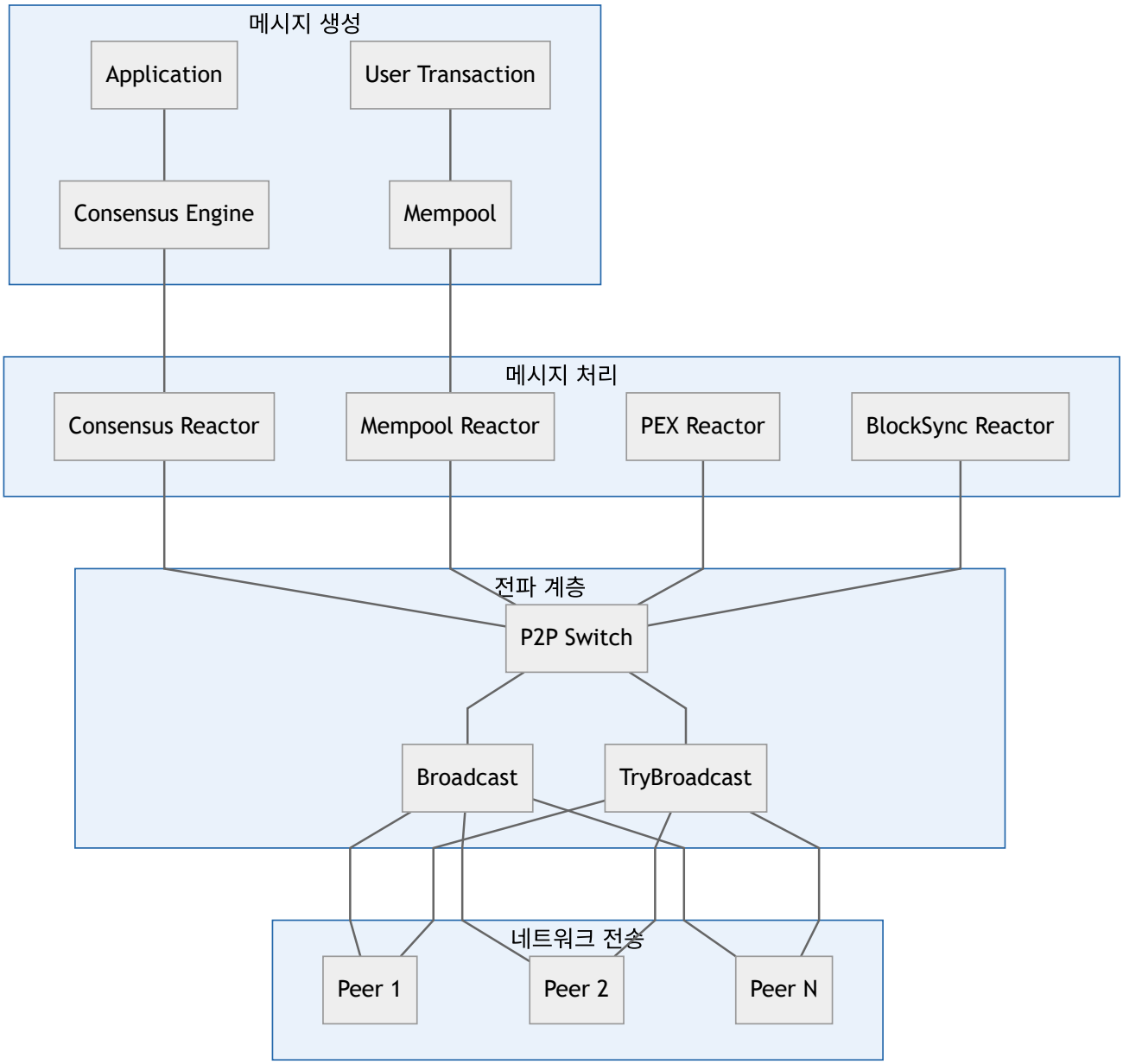
메시지 전파 및 가십 프로토콜

개요

CometBFT 네트워크에서 메시지 전파는 분산 시스템의 핵심 기능입니다. 블록, 트랜잭션, 투표 등의 중요한 정보가 네트워크 전체에 효율적이고 신뢰성 있게 전달되어야 합니다. 이 문서에서는 CometBFT의 메시지 전파 메커니즘과 가십 프로토콜의 동작 원리를 상세히 설명합니다.

메시지 전파 아키텍처

1. 전체 구조



2. 메시지 유형별 전파 방식

합의 메시지 (Consensus Messages)

- **Proposal**: 블록 제안 메시지
- **Prevote**: 사전 투표 메시지
- **Precommit**: 사전 커밋 메시지

- **BlockPart**: 블록 파트 메시지

트랜잭션 메시지 (Transaction Messages)

- **Tx**: 개별 트랜잭션
- **TxBatch**: 트랜잭션 배치 (최적화)

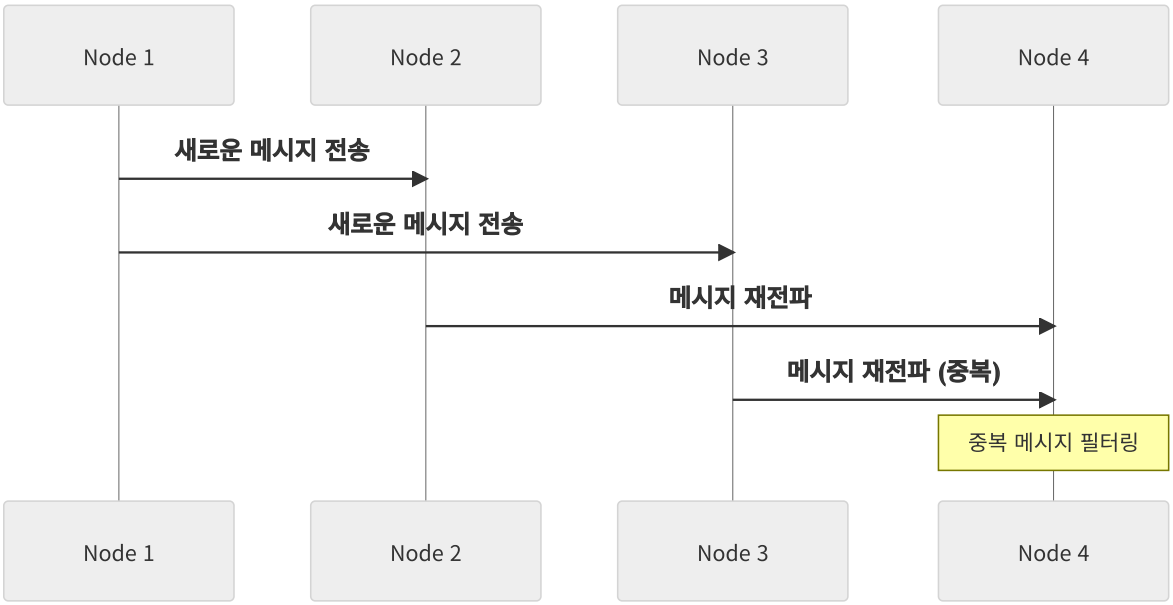
동기화 메시지 (Sync Messages)

- **BlockRequest**: 블록 요청
- **BlockResponse**: 블록 응답
- **StatusRequest**: 상태 요청
- **StatusResponse**: 상태 응답

가십 프로토콜 구현

1. 기본 가십 메커니즘

CometBFT는 **Push-based Gossip** 방식을 사용합니다:



2. 브로드캐스트 구현

Broadcast (블로킹 방식)

```
func (sw *Switch) Broadcast(e Envelope) {
_, _ = e.marshalMessage()
sw.peers.ForEach(func(p Peer) {
go  func(peer Peer) {
_ = peer.Send(e) // 블로킹 전송
}(p)
})
}
```

TryBroadcast (논블로킹 방식)

```go
func (sw *Switch) TryBroadcast(e Envelope) {
_, _ = e.marshalMessage()
sw.peers.ForEach(func(p Peer) {
go func(peer Peer) {
_ = peer.TrySend(e) // 논블로킹 전송
}(p)
})
}
```

3. 메시지 중복 제거

메시지 식별자

각 메시지는 고유한 식별자를 가집니다:

- **트랜잭션**: 트랜잭션 해시
- **블록**: 블록 해시 + 높이
- **투표**: 투표자 ID + 라운드 + 타입

중복 필터링 메커니즘

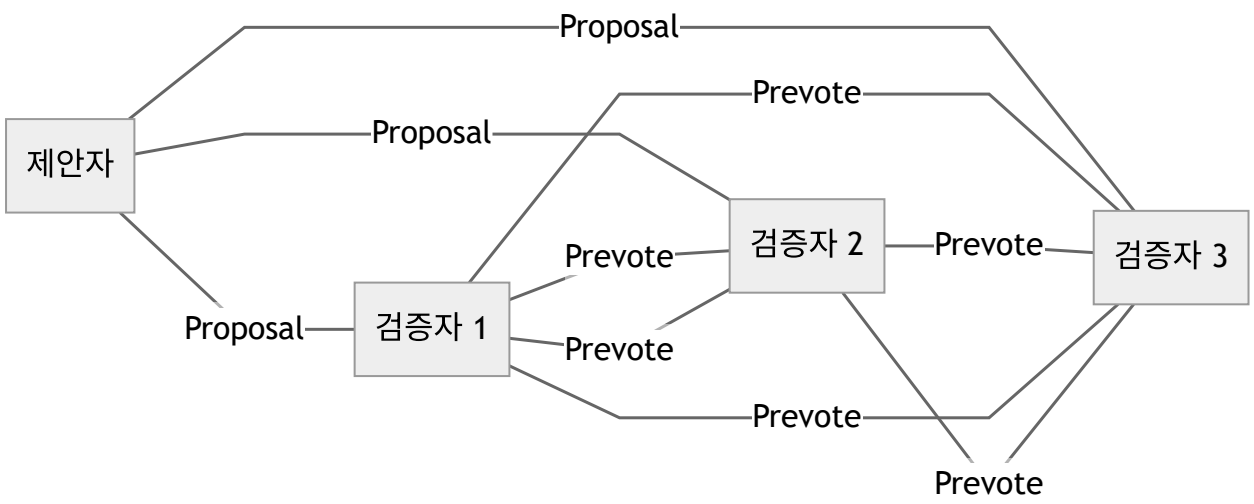
```go
type MessageCache struct {
cache map[string]time.Time
mutex sync.RWMutex
ttl time.Duration
}
```

```go
func (mc *MessageCache) HasSeen(msgID string) bool {
mc.mutex.RLock()
defer mc.mutex.RUnlock()
timestamp, exists := mc.cache[msgID]
if !exists {
return false
}
// TTL 확인
return time.Since(timestamp) < mc.ttl
}
```

메시지 유형별 전파 전략

1. 합의 메시지 전파

Proposal 메시지



투표 메시지 집계

```
type VoteSet struct {
votes  map[string]*types.Vote
mutex sync.RWMutex
}

func (vs *VoteSet) AddVote(vote *types.Vote) (bool, error) {
vs.mutex.Lock()
defer  vs.mutex.Unlock()
voteID := vote.ValidatorID + vote.Type.String()
if  _, exists := vs.votes[voteID];  exists {
return false, nil  // 중복 투표
}
vs.votes[voteID] = vote
return true, nil
}
```

2. 트랜잭션 전파

개별 트랜잭션 전파

```
func (memR *Reactor) BroadcastTx(tx types.Tx) error {
// 트랜잭션 검증
```

```go
if err := memR.mempool.CheckTx(tx); err != nil {
return err
}
// 네트워크에 브로드캐스트
envelope := p2p.Envelope{
ChannelID: MempoolChannel,
Message: &protomem.Txs{Txs: [][]byte{tx}},
}
memR.Switch.TryBroadcast(envelope)
return nil
}
```

배치 전파 최적화

```go
type TxBatch struct {
txs []types.Tx
maxSize int
maxDelay time.Duration
timer *time.Timer
}

func (tb *TxBatch) AddTx(tx types.Tx) {
tb.txs = append(tb.txs, tx)
if len(tb.txs) >= tb.maxSize {
```

```
tb.flush()
} else if tb.timer == nil {
tb.timer = time.AfterFunc(tb.maxDelay, tb.flush)
}
}
```

3. 블록 전파

블록 파트 시스템

큰 블록은 여러 파트로 나누어 전송됩니다:

```
type BlockPart struct {
Height int64
Round int32
Part *types.Part
}

func (bs *BlockSyncReactor) BroadcastBlock(block *types.Block) {
partSet := block.MakePartSet(types.BlockPartSizeBytes)
for i := 0; i < partSet.Total(); i++ {
part := partSet.GetPart(i)
envelope := p2p.Envelope{
ChannelID: BlockSyncChannel,
Message: &protobcr.BlockPart{
```

```
Height: block.Height,
Round: 0,
Part: part.ToProto(),
},
}
bs.Switch.TryBroadcast(envelope)
}
}
```

네트워크 최적화 기법

1. 적응적 전파 전략

네트워크 상태 기반 조절

```
type AdaptiveGossip struct {
networkLatency time.Duration
bandwidth  int64
peerCount  int
}

func (ag *AdaptiveGossip) SelectPeers(allPeers []Peer, fanout  int) []Peer {
// 네트워크 상태에 따른 fanout 조절
if  ag.networkLatency > 100*time.Millisecond {
```

```
fanout = fanout * 2  // 지연이 클 때 더 많은 피어에 전송
}
if  ag.bandwidth < 1024*1024 { // 1MB/s 미만
fanout = fanout / 2  // 대역폭이 낮을 때 전송 수 감소
}
return  ag.selectRandomPeers(allPeers, fanout)
}
```

피어 품질 기반 선택

```
type PeerQuality struct {
latency time.Duration
reliability  float64
bandwidth  int64
}

func (pq *PeerQuality) Score() float64 {
latencyScore := 1.0 / (1.0 + pq.latency.Seconds())
bandwidthScore := math.Log(float64(pq.bandwidth)) / 20.0
return (latencyScore + pq.reliability + bandwidthScore) / 3.0
}
```

2. 메시지 우선순위 시스템

우선순위 큐

```go
type PriorityMessage struct {
Message p2p.Envelope
Priority  int
Timestamp time.Time
}

type MessageQueue struct {
high  chan PriorityMessage
medium  chan PriorityMessage
low  chan PriorityMessage
}

func (mq *MessageQueue) Send() {
for {
select {
case  msg := <-mq.high:
mq.sendMessage(msg)
case  msg := <-mq.medium:
mq.sendMessage(msg)
case  msg := <-mq.low:
mq.sendMessage(msg)
```

```
}
}
}
```

메시지 유형별 우선순위

- **높음**: 합의 관련 메시지 (Proposal, Vote)
- **중간**: 블록 동기화 메시지
- **낮음**: 트랜잭션, PEX 메시지

3. 대역폭 관리

Rate Limiting

```go
type RateLimiter struct {
rate  int64  // bytes per second
bucket  int64  // current bucket size
lastTime time.Time
mutex sync.Mutex
}

func (rl *RateLimiter) Allow(size  int64) bool {
rl.mutex.Lock()
defer  rl.mutex.Unlock()
now := time.Now()
elapsed := now.Sub(rl.lastTime)
```

```
// 버킷 보충
rl.bucket += int64(elapsed.Seconds() * float64(rl.rate))
if  rl.bucket > rl.rate {
rl.bucket = rl.rate
}
if  rl.bucket >= size {
rl.bucket -= size
rl.lastTime = now
return true
}
return false
}
```

동적 대역폭 할당

```
func (sw *Switch) AllocateBandwidth() {
totalBandwidth := sw.config.MaxBandwidth
peerCount := sw.peers.Size()
if  peerCount == 0 {
return
}
// 기본 할당량
baseAllocation := totalBandwidth / int64(peerCount)
```

```go
// 우선순위 피어에 추가 할당
for _, peer := range sw.peers.List() {
if peer.IsPersistent() {
peer.SetBandwidthLimit(baseAllocation * 2)
} else {
peer.SetBandwidthLimit(baseAllocation)
}
}
}
```

신뢰성 보장 메커니즘

1. 메시지 확인 시스템

ACK 기반 확인

```go
type MessageTracker struct {
pending map[string]*PendingMessage
mutex sync.RWMutex
}

type PendingMessage struct {
Message p2p.Envelope
Timestamp time.Time
```

```go
Retries int
Peers map[string]bool // 전송된 피어들
}

func (mt *MessageTracker) TrackMessage(msgID string, msg p2p.Envelope) {
mt.mutex.Lock()
defer mt.mutex.Unlock()
mt.pending[msgID] = &PendingMessage{
Message: msg,
Timestamp: time.Now(),
Retries: 0,
Peers: make(map[string]bool),
}
}
```

재전송 메커니즘

```go
func (mt *MessageTracker) RetryPendingMessages() {
mt.mutex.Lock()
defer mt.mutex.Unlock()
now := time.Now()
for msgID, pending := range mt.pending {
if now.Sub(pending.Timestamp) > 5*time.Second {
if pending.Retries < 3 {
```

```
mt.retryMessage(msgID, pending)
pending.Retries++
pending.Timestamp = now
} else {
delete(mt.pending, msgID) // 포기
}
}
}
}
```

2. 네트워크 분할 대응

파티션 감지

```
type PartitionDetector struct {
expectedPeers  int
currentPeers  int
threshold  float64
}

func (pd *PartitionDetector) IsPartitioned() bool {
ratio := float64(pd.currentPeers) / float64(pd.expectedPeers)
return  ratio < pd.threshold
```

```
}
```

복구 전략

```
func (sw *Switch) HandlePartition() {
// 1. 시드 노드에 재연결 시도
sw.dialSeeds()
// 2. 저장된 피어 주소로 연결 시도
addrs := sw.addrBook.GetSelection()
for _, addr := range addrs {
go sw.DialPeerWithAddress(addr)
}
// 3. 백업 시드 노드 활용
if len(sw.backupSeeds) > 0 {
sw.dialBackupSeeds()
}
}
```

성능 모니터링 및 메트릭

1. 전파 성능 메트릭

지연 시간 측정

```
type PropagationMetrics struct {
```

```
messageLatency prometheus.HistogramVec

fanoutSize prometheus.GaugeVec

duplicateRate prometheus.CounterVec

}


func (pm *PropagationMetrics) RecordLatency(msgType  string, latency time.Durati

pm.messageLatency.WithLabelValues(msgType).Observe(latency.Seconds())

}
```

처리량 모니터링

```
type ThroughputMonitor struct {

bytesPerSecond prometheus.GaugeVec

messagesPerSecond prometheus.GaugeVec

window time.Duration

samples []Sample

}


type Sample struct {

timestamp time.Time

bytes  int64

messages  int64
```

```
}
```

2. 네트워크 건강성 지표

연결성 메트릭

- **피어 수**: 현재 연결된 피어 수
- **메시지 전파 시간**: 메시지가 네트워크 전체에 전파되는 시간
- **중복률**: 중복 메시지 비율
- **손실률**: 메시지 손실 비율

대시보드 예시

```
{
"metrics": {
"peer_count": 45,
"avg_propagation_time": "150ms",
"duplicate_rate": "5.2%",
"message_loss_rate": "0.1%",
"bandwidth_utilization": "67%"
}
}
```

문제 해결 및 최적화

1. 일반적인 문제들

메시지 전파 지연

```
# 원인 분석
grep "message propagation" ~/.cometbft/logs/cometbft.log
curl http://localhost:26657/net_info | jq '.result.peers'

# 해결 방법
# 1. 피어 수 증가
# 2. 네트워크 대역폭 확인
# 3. 메시지 크기 최적화
```

중복 메시지 과다

```
# 중복률 확인
curl http://localhost:26657/metrics | grep duplicate

# 최적화 설정
[p2p]
flush_throttle_timeout = "10ms"
max_packet_msg_payload_size = 1024
```

2. 성능 튜닝

네트워크 설정 최적화

```
[p2p]

# 전송 속도 제한

send_rate = 5120000 # 5MB/s

recv_rate = 5120000 # 5MB/s



# 메시지 크기 제한

max_packet_msg_payload_size = 1024



# 플러시 간격

flush_throttle_timeout = "100ms"



# 피어 수 제한

max_num_outbound_peers = 10

max_num_inbound_peers = 40
```

메모리 사용량 최적화

```
[mempool]

# 메모리풀 크기 제한

size = 5000

max_txs_bytes = 1073741824 # 1GB



# 캐시 설정
```

```
cache_size = 10000
```

이러한 메시지 전파 및 가십 프로토콜을 통해 CometBFT는 효율적이고 신뢰성 있는 분산 통신을 구현하며, 네트워크 전체의 일관성과 성능을 보장합니다.